

Adaptive Route Planning

using Pathfinding Algorithms

Comparing Random · DFS · BFS · Dijkstra · A*

Anna Hyunjung Kim

Problem Description

The environment is a 2D grid where agents must navigate from

S (Start) to **G (Goal)**

under varying terrain conditions.

Unlike simple shortest-path problems, this setup introduces different movement costs — making navigation more realistic and challenging.



```
grid[0][0] = "S"
grid[size - 1][size - 1] = "G"

for i in range(size):
    for j in range(size):
        if grid[i][j] == ".":
            r = random.random()

            if r < 0.1:
                grid[i][j] = "#"
            elif r < 0.15:
                grid[i][j] = "~"
            elif r < 0.2:
                grid[i][j] = "X"
```



Start Point



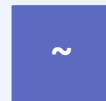
Goal (Destination)



Normal Road (cost 1)



Wall (blocked)



Slow Region (cost 3)



Traffic light (cost 5)

Algorithms Compared

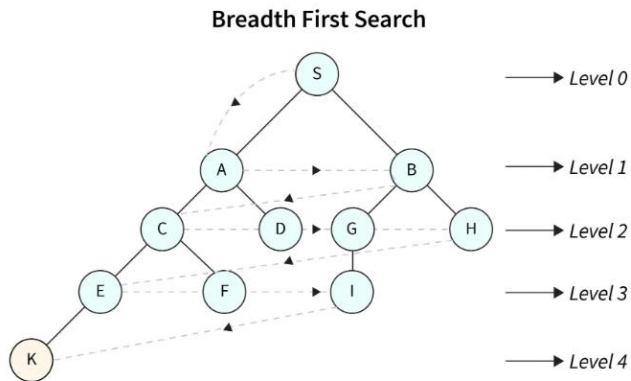
Random	DFS	BFS	Dijkstra	A*
Baseline	Stack-based	Queue-based	Cost-aware	Heuristic
Random moves No planning	Deep exploration Not optimal	Shortest steps Ignores cost	Lowest total cost No heuristic	Cost + Heuristic Most efficient
Optimal: X	Optimal: X	Optimal: Steps	Optimal: Cost	Optimal: Best

How They Work: Key Algorithms

BFS — Breadth-First Search

Structure: Queue (FIFO)

Guarantee: Shortest path (steps)



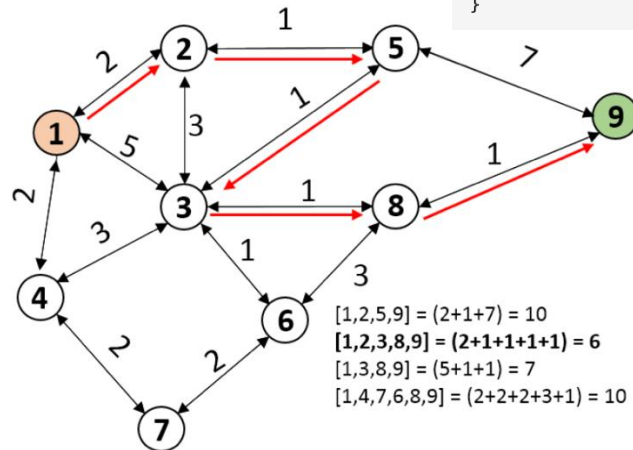
<https://pages.cs.wisc.edu/~ariana/>

Dijkstra's Algorithm

Structure: Priority Queue

Guarantee: Lowest total cost

```
cost_map = {  
    ".": 1,  
    "S": 1,  
    "G": 1,  
    "~": 3,  
    "X": 5,  
    "#": float("inf")  
}
```



A* Search — Best of Both Worlds

$$f(n) = g(n) + h(n)$$

$g(n)$ — Cost So Far

The actual accumulated cost from start to current node.
Same as Dijkstra's distance.

$h(n)$ — Heuristic (Manhattan)

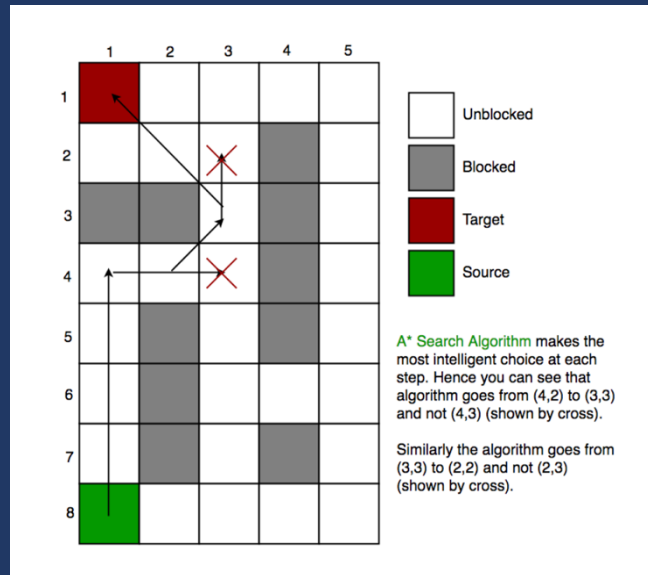
Estimated cost to goal:
 $h(n) = |x_{\text{cur}} - x_{\text{goal}}| + |y_{\text{cur}} - y_{\text{goal}}|$

Prioritizes directions toward the goal

Reduces unnecessary exploration

Faster than Dijkstra in practice

Guarantees optimal cost (with admissible heuristic)

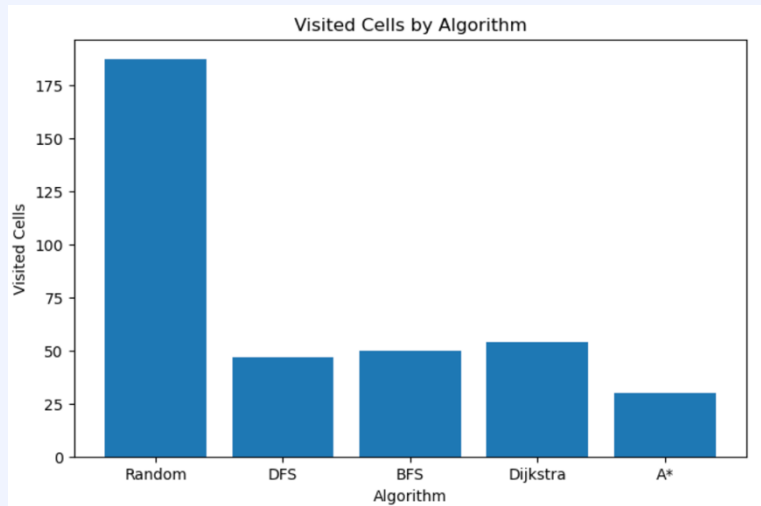


Results & Observations

Algorithm	Optimal?	Cost-Aware?	Heuristic?	Speed	Best For
Random	X	X	X	N/A	Baseline only
DFS	X	X	X	Fast	Quick path (may be long)
BFS	O	X	X	Medium	Equal-cost grids
Dijkstra	O	O	X	Slow	Variable terrain costs
A*	O	O	O	Fastest	General use (recommended)

Implementation & Visualization

	Algorithm	Success	Steps	Path Length	Visited Cells	Runtime	Total Cost	Map Size
0	BFS	False	0	0	3	0.00002	NaN	4
1	Dijkstra	True	6	6	9	0.00004	10.0	4
2	A*	True	6	6	8	0.00003	10.0	4
3	BFS	False	0	0	2	0.00001	NaN	6
4	Dijkstra	False	0	0	2	0.00001	NaN	6
5	A*	False	0	0	2	0.00001	NaN	6
6	BFS	True	14	14	49	0.00008	NaN	8
7	Dijkstra	True	14	14	58	0.00012	14.0	8
8	A*	True	14	14	31	0.00008	14.0	8
9	BFS	False	0	0	2	0.00001	NaN	10
10	Dijkstra	True	18	18	90	0.00020	22.0	10
11	A*	True	18	18	47	0.00012	22.0	10



Implementation & Visualization

Adaptive Delivery AI Simulator

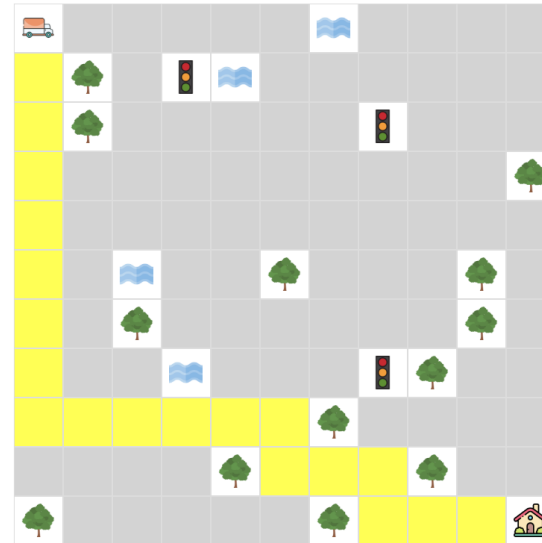
Map Size

4

Algorithm

BFS

Run



Conclusion

Simpler algorithms: focus on minimizing distance (steps)

Advanced algorithms: incorporate terrain cost for smarter routing

Heuristic methods (A*): improve efficiency without sacrificing optimality

Real-world relevance: autonomous vehicles, logistics, robot navigation

Challenge: Understanding Algorithms. Put image to Streamlit

Next: More Algorithms